

GUIDE DÉVELOPPEUR

Pipeline Kubernetes

Arborescence du projet et inventaire technique des fichiers

Objectif du document

Ce guide présente l'organisation du dépôt du pipeline Kubernetes. Il sert de référence rapide pour comprendre le rôle des principaux dossiers, manifestes Kubernetes, volumes persistants et scripts d'automatisation.

Ce document complète le guide utilisateur : il ne décrit pas les commandes d'usage courant, mais la structure technique du projet et les fichiers à connaître avant de modifier ou maintenir le pipeline.

Table des matières

Table des matières.....	1
1. Arborescence.....	2
2. Inventaire des fichiers.....	2
2.1 Kafka.....	2
2.2 Pipeline batch.....	3
2.3 Pipeline incrémental.....	3
2.4 Volumes persistants.....	4
2.5 Scripts d'automatisation.....	4
2.6 Services et permissions.....	4
3. Points d'attention.....	4
Checklist.....	5

1. Arborescence

L'arborescence du projet sépare le code Python original, les éléments Docker et les ressources Kubernetes. Le dossier k8s regroupe la majorité des fichiers nécessaires à l'exécution du pipeline sur le cluster.

Chemin	Rôle
code/	Code Python du pipeline original.
code/Energy-Aware-Entity-Resolution/distribution/	Scripts d'adaptation et de distribution du pipeline.
docker/	Images Docker utilisées pour exécuter les pods du pipeline.
k8s/	Manifestes Kubernetes et scripts d'automatisation.
k8s/kafka/	Manifestes liés au serveur Kafka/Redpanda.
k8s/pipeline/batch/	Manifeste du workflow Argo utilisé pour les tâches batch.
k8s/pipeline/incremental/	Manifestes des pods de la partie incrémentale du pipeline.
k8s/pipeline/exec/	État compilé des manifestes, prêt à être créé ou appliqué sur le cluster.
k8s/pvc-manifests/	Manifestes des volumes persistants.
k8s/scripts/	Scripts d'automatisation et wrapper de commandes.
k8s/svc-manifests/	Manifestes des services Kubernetes.

```
code/
|—— Energy-Aware-Entity-Resolution/
|   |—— distribution/
docker/
k8s/
|—— kafka/
|—— pipeline/
|   |—— batch/
|   |—— incremental/
|   |—— exec/
|—— pvc-manifests/
|—— scripts/
|—— svc-manifests/
```

À retenir

Le dossier k8s/pipeline/exec/ correspond à l'état compilé des manifestes. C'est une sortie générée à partir de la configuration et des manifestes sources.

2. Inventaire des fichiers

Les fichiers ci-dessous sont regroupés par rôle afin de faciliter la maintenance du projet. Les chemins sont indiqués relativement à la racine du dépôt.

2.1 Kafka

Fichier	Rôle
k8s/kafka/kafka-server-config.yaml	ConfigMap injectant la configuration dans le pod serveur Kafka.
k8s/kafka/kafka-server.yaml	Pod créant un serveur Kafka basé sur l'image Redpanda.

2.2 Pipeline batch

Fichier	Rôle
k8s/pipeline/batch/pipeline.yaml	Workflow Argo regroupant les tâches BERT et les tâches batch du mode embedding.

2.3 Pipeline incrémental

Fichier	Rôle
k8s/pipeline/incremental/calculating_similarity.yaml	Pod lançant la tâche calculating similarity en mode incrémental.
k8s/pipeline/incremental/candidate_enumeration.yaml	Pod lançant la tâche candidate enumeration en mode incrémental.
k8s/pipeline/incremental/cg_feature_extraction.yaml	Pod lançant la tâche cg feature extraction en mode incrémental.
k8s/pipeline/incremental/consumer.yaml	Pod lançant le consumer Kafka.
k8s/pipeline/incremental/decision_making.yaml	Pod lançant la tâche decision making en mode incrémental.
k8s/pipeline/incremental/embedding_training.yaml	Pod lançant la tâche embedding training en mode incrémental.
k8s/pipeline/incremental/evaluation.yaml	Pod lançant la tâche evaluation.
k8s/pipeline/incremental/feature_index_construction.yaml	Pod lançant la tâche feature index construction en mode incrémental.
k8s/pipeline/incremental/graph_construction.yaml	Pod lançant la tâche graph construction en mode incrémental.
k8s/pipeline/incremental/init.yaml	Pod initialisant le pipeline.
k8s/pipeline/incremental/normalization.yaml	Pod lançant la tâche normalization en mode incrémental.
k8s/pipeline/incremental/producer.yaml	Pod lançant le producer Kafka.
k8s/pipeline/incremental/random_walk.yaml	Pod lançant la tâche random walk en mode incrémental.

2.4 Volumes persistants

Fichier	Rôle
k8s/pvc-manifests/pvc-bert-model.yaml	PVC stockant le modèle BERT.
k8s/pvc-manifests/pvc-cg-feature-index.yaml	PVC stockant l'index CG feature.
k8s/pvc-manifests/pvc-decision-evaluation.yaml	PVC stockant le graphe de similarité.
k8s/pvc-manifests/pvc-graph.yaml	PVC stockant le graphe.
k8s/pvc-manifests/pvc-reports.yaml	PVC stockant les résultats CodeCarbon.
k8s/pvc-manifests/pvc-buffer.yaml	PVC stockant les fichiers de buffer du mode incrémental.
k8s/pvc-manifests/pvc-data.yaml	PVC stockant le ou les jeux de données.
k8s/pvc-manifests/pvc-embedding-model.yaml	PVC stockant le modèle Embedding.
k8s/pvc-manifests/pvc-kafka-data.yaml	PVC stockant les données du serveur Kafka.

2.5 Scripts d'automatisation

Fichier	Rôle
k8s/scripts/compiler.py	Script compilant le fichier de configuration de répartition des pods.
k8s/scripts/configmaps.sh	Script créant les ConfigMaps des scripts Python et des fichiers de configuration.
k8s/scripts/erctl	Wrapper autour du script erctl.sh.
k8s/scripts/erctl.sh	Script principal regroupant les autres scripts sous une même commande.
k8s/scripts/fetch-codecarbon.sh	Script téléchargeant le rapport CodeCarbon sur la machine locale.
k8s/scripts/images.sh	Script gérant les images Docker du projet.
k8s/scripts/move.py	Script déplaçant les pods d'un nœud à l'autre.
k8s/scripts/pipeline.sh	Script permettant de manipuler le pipeline : démarrage, arrêt et suppression.
k8s/scripts/process.sh	Script récupérant les processus des tâches du pipeline et réalisant les mesures.
k8s/scripts/scheduling.yaml	Fichier de configuration de la répartition des pods sur les nœuds du cluster.
k8s/scripts/sync-data-pvc.sh	Script synchronisant le jeu de données avec les volumes du cluster.

2.6 Services et permissions

Fichier	Rôle
k8s/svc-manifests/kafka-server.yaml	Service exposant le serveur Kafka aux pods du cluster.
k8s/rbac-configmaps.yaml	Manifeste RBAC autorisant la manipulation de ressources du cluster et définissant le rôle associé.

3. Points d'attention

Avant modification

Modifier en priorité les fichiers sources : manifestes dans k8s/pipeline/batch/ ou k8s/pipeline/incremental/, configuration dans k8s/scripts/scheduling.yaml, puis régénérer l'état compilé si nécessaire.

Cohérence des chemins

Lorsqu'un chemin de jeu de données, de PVC ou de script change, vérifier les fichiers dépendants afin d'éviter une incohérence entre configuration, manifestes et scripts d'initialisation.

Nommage

Conserver des noms de fichiers explicites et cohérents avec les tâches du pipeline. Cela facilite la lecture des logs Kubernetes, des workflows Argo et des scripts d'automatisation.

Checklist

Modification	Fichiers à vérifier
Jeu de données	config/examples/, k8s/scripts/sync-data-pvc.sh, manifestes du pipeline concernés.
Répartition des pods	k8s/scripts/scheduling.yaml, k8s/scripts/compiler.py, k8s/pipeline/exec/.
Kafka	k8s/kafka/, k8s/svc-manifests/kafka-server.yaml, pods producer/consumer.
Volumes persistants	k8s/pvc-manifests/ et manifestes des pods qui montent ces volumes.
Mesures	k8s/scripts/process.sh et k8s/pvc-manifests/pvc-reports.yaml.